
CS772 Project Report

Chaitanya Bramhapurikar
230305

Naveen Kumar
220699

Riyanshi
220903

Shubhanshu Mishra
221048

Abstract

"This paper explores the integration of AutoElicit in active learning and deep learning. We demonstrate its ability to enhance sentiment classification by generating synthetic samples and improving sample selection with LLM-informed priors. Additionally, we show how AutoElicit can be used to initialize deep learning models (MLPs, CNNs, LSTMs) with meaningful priors, leading to faster, more stable learning with fewer examples."

1 Problem Description

Modern machine learning systems often require large amounts of labeled data to achieve high performance, making them inefficient and impractical in scenarios with limited data availability. While deep learning models such as CNNs and LSTMs have achieved remarkable success across tasks, they are particularly data-hungry and struggle in low-resource settings. At the same time, active learning strategies, which aim to select the most informative samples for labeling, are limited by their reliance on heuristic criteria like uncertainty or diversity, which may not always align with domain knowledge.

Recent advancements in large language models (LLMs) have introduced new ways to incorporate prior knowledge into learning processes through tools like AutoElicit, which can generate structured priors for downstream tasks. However, it remains unclear how to best integrate these LLM-generated priors into deep learning workflows and active learning pipelines to improve sample efficiency, model generalization, and training stability in low-data regimes.

This research seeks to address the dual challenge of (1) enhancing active learning strategies using AutoElicit-generated priors to guide data selection more effectively and (2) incorporating these priors into deep learning models to improve performance when data is scarce.

By evaluating the impact of LLM-informed priors on both the data acquisition process and model training dynamics, this work aims to establish a principled framework for leveraging LLMs in real-world low-resource learning scenarios.

2 Seed Paper Analysis : *AutoElicit: Using Large Language Models for Expert Prior Elicitation in Predictive Modelling*

The paper introduced AutoElicit, a framework that utilizes Large Language Models (LLMs) to automate the process of prior elicitation, i.e., encoding domain knowledge into predictive models without the need for manual expert input. The motivation behind this approach is to reduce the reliance on large labeled datasets by front-loading models with structured prior knowledge derived from LLMs.

AutoElicit outputs various types of priors, including feature importances, causal relations, label likelihoods, and logical rules or constraints. The framework positions itself as a bridge between traditional Bayesian modeling and modern data-driven methods, leveraging the general knowledge encoded in LLMs.

AutoElicit provides LLM-generated structured priors that can guide sample selection—offering a way to prioritize data points based on task-relevant knowledge, rather than relying solely on heuristics such as uncertainty or diversity. This makes it a promising tool for enhancing sample selection in active learning by integrating domain expertise into the process.

2.1 Relevance to Our Idea: Active Learning with AutoElicit

In the context of our work on active learning, AutoElicit provides a unique approach to selecting data samples. Rather than relying purely on uncertainty (which may introduce model bias) or diversity, AutoElicit enables us to prioritize samples based on structured knowledge generated by LLMs. This could help in identifying the most informative samples from the data, particularly in scenarios where human expertise is scarce or costly.

Incorporating AutoElicit’s priors into active learning strategies allows us to refine our query functions, ensuring that the samples chosen are not just uncertain but also aligned with the task-specific knowledge encoded in the priors. This can lead to more effective sample acquisition, especially in data-scarce environments, by guiding the model to focus on data points that are expected to contribute the most to improving the predictive model.

2.2 Relevance to Our Idea: Extending AutoElicit to Deep Learning Models

We’re taking AutoElicit beyond logistic regression and plugging its LLM-informed priors straight into modern neural nets—MLPs, CNNs, and LSTMs alike. Instead of starting from scratch with random weights or generic pre-training, our models begin life already tuned to the key patterns and color/shape hints an LLM distilled from a simple prompt.

On CIFAR-10, MNIST, or IMDB sentiment, this means faster, more reliable learning with far fewer examples. Training feels smoother (the loss hardly shakes), and you will not have to hunt for that perfect learning rate or batch size, our priors gently steer the network toward what really matters for the task. In short, AutoElicit gives deep models a head start, focusing them on the right features from day one rather than leaving them to “figure it out” purely from data.

3 IDEA 1: Active Learning with AutoElicit

3.1 Dataset Overview

For our experiments, we use the Stanford Sentiment Treebank 2 (SST-2) dataset, a widely adopted benchmark in natural language processing for binary sentiment classification. The dataset consists of short sentences extracted from movie reviews, each labeled as either expressing a positive or negative sentiment.

SST-2 is particularly well-suited for evaluating active learning strategies due to its compact sentence structure, clear labeling, and balanced class distribution. We focus on the binary version of the dataset, which excludes neutral labels to simplify the task. In total, the dataset includes approximately 8,500 labeled training samples, along with a separate validation and test set. Its manageable size and linguistic variety make it ideal for testing data-efficient learning approaches in low-resource settings.

3.2 Experimental results

3.2.1 Experiment 1: Baseline Evaluation Using a Pretrained DistilBERT Model

🔗 In this experiment, we implemented a straightforward baseline by evaluating a pretrained DistilBERT model fine-tuned on the SST-2 dataset, without applying any active learning strategies. The objective was to establish a reference point for sentiment classification performance when using a large-scale language model directly out of the box.

We loaded the validation split of the SST-2 dataset from the GLUE benchmark and used the Hugging Face Transformers and Datasets libraries to process and evaluate the data. The model was run in inference mode to predict sentiment labels, and the predictions were compared with the ground truth labels to calculate overall accuracy.

In addition to computing accuracy, we inspected a few example predictions to qualitatively assess the model’s outputs. This baseline serves as a control to better contextualize the gains achieved through active learning and prior integration in subsequent experiments.

3.2.2 Experiment 2: Active Learning with Naive Random Acquisition

🔗 In this experiment, we implemented a naive active learning loop using random acquisition as a baseline approach. The goal was to assess the effectiveness of active learning when new training samples are selected at random, without any uncertainty or disagreement-based criteria.

We began with a small set of 50 labeled samples from the SST-2 training data. In each iteration of the loop, the model (DistilBERT fine-tuned for binary sentiment classification) was trained on the currently labeled set. Following training, the model was evaluated on the SST-2 validation set, and the classification accuracy was recorded.

After evaluation, a batch of new samples was randomly selected from the unlabeled pool and added to the labeled set for the next iteration. This cycle was repeated over 10 iterations, incrementally increasing the labeled data size by 50 samples each round. The performance trend was visualized by plotting validation accuracy against the number of labeled samples.

This experiment serves as a comparative benchmark to gauge the impact of intelligent sampling strategies, such as uncertainty sampling and query-by-committee, used in other experiments.

3.2.3 Experiment 3: Active Learning with Uncertainty Sampling

🔗 In this experiment, we implemented uncertainty sampling, a fundamental strategy in active learning where the model selects the most uncertain unlabeled examples for annotation. The underlying assumption is that these ambiguous samples, which typically lie near the model’s decision boundary, are the most informative for improving performance.

We fine-tuned a DistilBERT model on an initial labeled subset of 50 samples from the SST-2 dataset. In each iteration, we evaluated the model’s confidence across the unlabeled pool using predictive entropy. The 50 samples with the highest entropy were then selected for labeling and added to the training set. The model was retrained from scratch after each acquisition to ensure consistent evaluation.

Validation accuracy was tracked after each round, providing insight into how efficiently the model learns from selectively acquired data. This approach serves as a baseline for comparing the effectiveness of more complex strategies such as Query by Committee.

3.2.4 Experiment 4: Active Learning with Query by Committee (QBC)

🔗 In this experiment, we implement active learning using the Query by Committee (QBC) strategy. The core idea is to train multiple models (a committee) on bootstrap samples of the labeled data, and use their disagreement on unlabeled data to select new informative samples for labeling. The intuition is that if committee members disagree on a sample’s label, that sample is likely to be informative and worth labeling.

We use a committee of three DistilBERT models, each trained on a different resampled version of the currently labeled set. For each active learning iteration, we compute vote entropy over the committee predictions for the unlabeled pool. Vote entropy quantifies the uncertainty or disagreement within the committee—samples with the highest vote entropy are selected for labeling and added to the labeled set.

Model performance is evaluated using majority voting over the committee on the validation set. This setup simulates a realistic scenario where model consensus plays a key role in decision-making. The process is repeated over multiple acquisition iterations, and we track the improvement in validation accuracy as the number of labeled samples increases.

3.2.5 Experiment 5: Active Learning with LLM-Generated Samples via AutoElicit

✎ In this experiment, we implemented an enhanced active learning pipeline by integrating synthetic sample generation using a Large Language Model (LLM) through the AutoElicit framework. This experiment serves as the core contribution of our study, aiming to address the bottleneck of limited labeled data in early active learning stages by augmenting the training pool with high-quality synthetic examples.

Motivation While previous experiments explored traditional active learning paradigms—such as evaluation-only baselines, naive random acquisition, and iterative fine-tuning—these approaches rely exclusively on the limited pool of human-annotated data. To further improve data efficiency, we propose augmenting the labeled dataset with synthetic examples generated via an LLM (in our case, DeepSeek-R1 via the ollama interface), guided by real SST-2 training examples. This approach leverages in-context learning to synthesize high-fidelity sentiment classification data and is particularly effective in low-resource active learning scenarios.

Methodology The experiment proceeded in two distinct phases:

a) Naive Acquisition Phase (Steps 1–5): We initialized the active learning loop by randomly selecting 20 labeled examples from the SST-2 training set. In each of the five iterations, 20 additional examples were randomly acquired and added to the training set, followed by fine-tuning a DistilBERT model on the current labeled pool. Validation accuracy was recorded after each step using the SST-2 validation set.

b) Synthetic Expansion Phase (LLM Augmentation): After the initial acquisition steps, we used AutoElicit to prompt a powerful LLM to generate 60 new training examples. Each generation round used 10 in-context examples (randomly sampled from the current labeled pool) and requested 3 novel sentence–label pairs that matched the sentiment style and linguistic structure of SST-2. The generated outputs were parsed, cleaned, and appended to the existing training data. The DistilBERT model was then re-trained using both the original and synthetic examples, and the final validation accuracy was recorded.

3.3 Learnings

Based on the results from the five experiments, we can draw several important conclusions about the effectiveness of different active learning strategies and the use of synthetic data generation with LLM in improving model performance. The following are the key findings, supported by the quantitative results:

a) Pre-trained Model Evaluation (Baseline):

When evaluating the DistilBERT model directly on the SST-2 dataset without any active learning, the model achieved an accuracy of 50.92%. This serves as our baseline performance, demonstrating the need for improvement through active learning methods.

b) Random Sampling:

With random sampling for active learning, the model’s performance significantly improved, reaching a final accuracy of 80.39% after 10 acquisition rounds (50 samples per round). This suggests that even with naive random selection of samples, the model benefits from additional labeled data.

c) Uncertainty Sampling:

Using uncertainty sampling, the model acquired the least confident samples in each round. This approach led to a final validation accuracy of 82.91%, which is slightly lower than the performance achieved by random sampling but still indicates a solid improvement over the baseline.

d) Query by Committee (QBC):

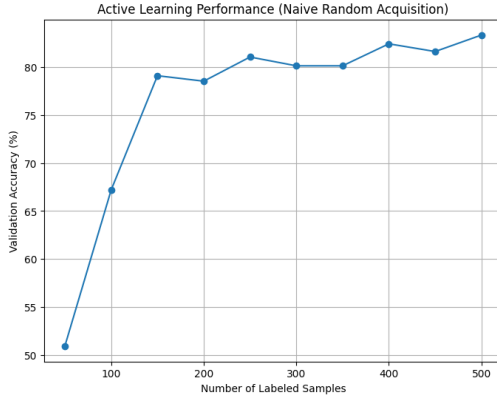
The Query by Committee (QBC) method, which uses multiple models to vote on the most uncertain samples, resulted in a final validation accuracy of 83.37%. This method performed similarly to random sampling, suggesting that QBC is effective but does not significantly outperform random selection in this particular task.

e) LLM-based Active Learning (Autoelicit):

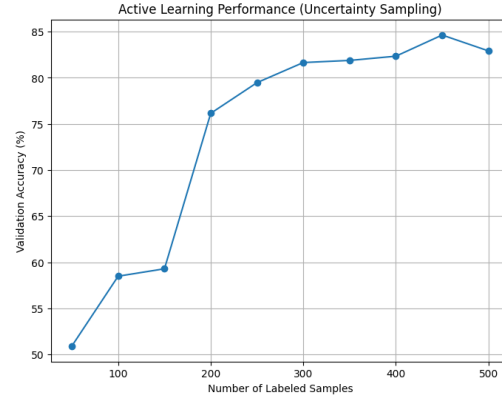
In the final experiment using LLM-based synthetic data generation (Autoelicit), the model achieved 82.22% accuracy with just around 250 labeled samples, which is comparable to the performance achieved by random sampling, uncertainty sampling, and QBC after acquiring much larger numbers of labeled samples. This indicates that synthetic data generation can significantly reduce the number of required labeled samples while achieving similar or slightly lower performance.

Conclusion The key takeaway is that LLM-based synthetic data generation (Autoelicit) is a highly effective method in reducing the need for large manually labeled datasets while maintaining good accuracy, even when compared to traditional active learning strategies like random sampling, uncertainty sampling, and QBC.

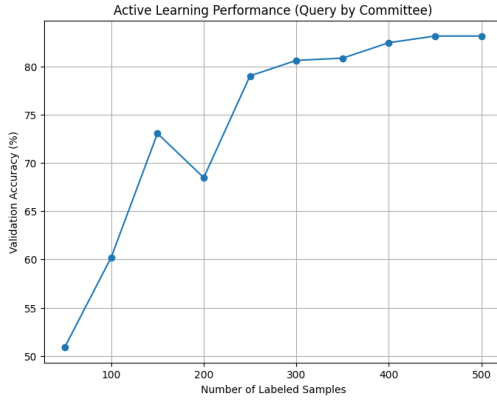
Although active learning strategies (e.g., random sampling, uncertainty sampling, and QBC) provided solid improvements over the baseline, the use of synthetic data helped achieve similar performance with fewer labeled samples, demonstrating the potential of LLMs in active learning tasks.



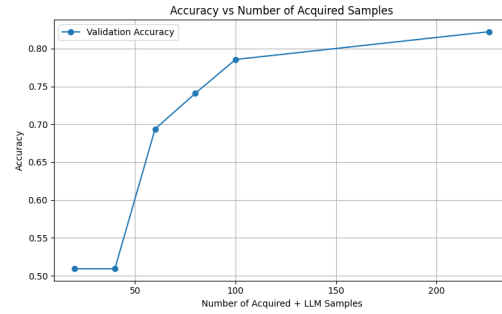
(a) Experiment 2: Naive random acquisition



(b) Experiment 3: Uncertainty sampling



(c) Experiment 4: Query-by-committee



(d) Experiment 5: LLM-augmented acquisition (AutoElicit)

Figure 1: Comparison of active learning strategies on SST-2. Each plot shows validation accuracy as a function of the number of labeled (and/or synthetic) samples.

3.4 Limitations and future research

Experiment overview : LLM-Augmented Active Learning with Uncertainty-Guided Prompts

We performed another experiment, in this setup, we begin with a small labeled subset (50 samples) from the SST-2 dataset and incrementally add new labeled samples through random acquisition over 10 active learning iterations. After each acquisition round and training, the model identifies the most uncertain validation samples using entropy-based uncertainty estimation.

These uncertain samples are then used to construct prompts for an LLM to generate additional synthetic training data. This LLM-generated data is added to the training loop over 10 augmentation epochs to assess whether it enhances performance. Validation accuracy is monitored throughout to evaluate the effect of synthetic data on generalization.

Observations The performance trend of the model under LLM-augmented training using the AutoElicit method reveals that while early phases show promising gains, the approach lacks long-term stability. For example, validation accuracy improved from 80.28% in Epoch 1 to a peak of 83.37% in Epoch 7, indicating the initial effectiveness of LLM-generated samples. However, in the absence of semantic filtering or human oversight, performance began to deteriorate as the quality of synthetic examples declined. By Epoch 9 and 10, accuracy dropped to 75.69% and 71.56%, respectively. This decline suggests that LLM bias and drift in content quality can accumulate over time, ultimately degrading model performance. These results highlight that while AutoElicit is useful in the short term, it becomes unstable without mechanisms to ensure the semantic relevance and polarity accuracy of the generated data.

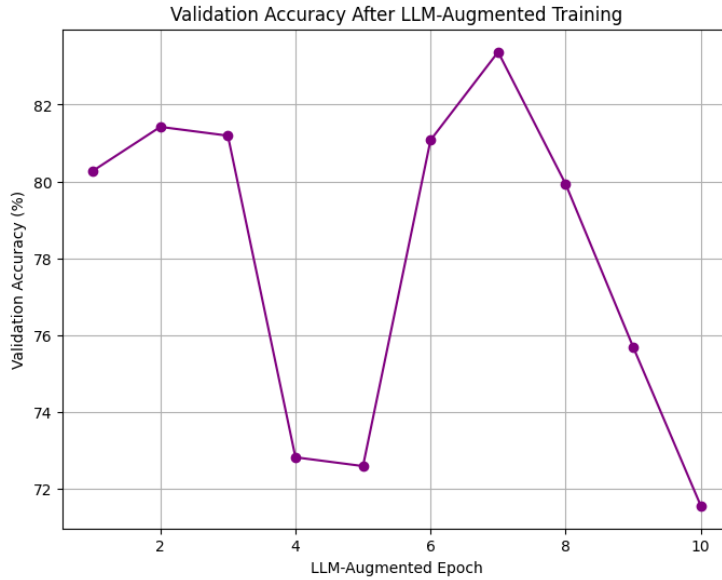


Figure 2: Validation accuracy across LLM-augmented training epochs

4 Idea 2 : AutoElicit for Prompt Optimization in Few/Zero-Shot Learning

We have thought of a method to use AutoElicit as a meta-prompting tool to potentially improve the performance of downstream large language models (LLMs) in few-shot and zero-shot settings. While we do not aim to modify the internal behavior of a target LLM, we hypothesize that improving the quality of prompts can lead to better outputs, even when using the same underlying model. The idea is as follows: in a zero-shot setting, a baseline prompt is first created and given to an LLM, and its output is noted. Instead of using this prompt directly for further inference, we would then input the same baseline prompt into AutoElicit and request a set of improved or alternative prompts. These generated prompts can then be fed into another LLM (or even the same one) to observe whether the response quality improves. In the few-shot scenario, AutoElicit can similarly be used to enhance example formatting, ordering, or phrasing. Though we have not yet implemented this pipeline experimentally, we believe it could provide a promising direction for cross-model prompt enhancement, where one LLM (via AutoElicit) indirectly improves the performance of another through prompt optimization.

5 Idea 3: Deep Learning Models with AutoElicit

5.1 Dataset Overview

CIFAR-10 is a classic vision benchmark of 60 000 tiny (32×32) color images across 10 everyday object classes. We train on 50 000 images and test on 10 000, striking a balance between speed and diversity. Its small size lets us iterate quickly, while the variety of classes—from airplanes to frogs—makes it perfect for gauging how well LLM-informed priors or active learning strategies help a model learn with fewer examples.

IMDB Sentiment features 50 000 real movie reviews split evenly into training and test sets, each labeled positive or negative. For our experiments, we use 40 000 reviews to train and 10 000 to test. The reviews range from short quips to long analyses, offering rich language patterns that challenge sentiment models. Its mix of lengths and writing styles makes IMDB ideal for seeing how LLM-elicited insights can speed up learning and boost performance with less labeled data.

5.2 Experimental results

5.2.1 Experiment 1: Baseline CNN with Random Initialization

🔗 We trained a lightweight CNN from scratch on CIFAR-10—no pretraining or LLM priors, just PyTorch’s default Kaiming random weights. Inputs underwent 32×32 random crops (±4px padding), flips, and normalization; test images were normalized only. Our SimpleCNN has two Conv–BN–ReLU blocks (32→64 channels) each with 2×2 max-pooling, followed by two fully connected layers. We ran five experiments using 20%, 40%, 60%, 80% and 100% of the 50 000-image training set, training 10 epochs per split with Adam (lr=1e-3) and cross-entropy loss. Each model was then evaluated on the full 10 000-image test split. This gives a clear “training data fraction vs. test accuracy” curve, establishing our no-frills CNN baseline.

5.2.2 Experiment 2: Improved CNN with LLM-Elicited Priors

🔗 In this experiment, we inject structured, task-specific knowledge from an LLM directly into our CNN before any training begins. We prompt a local LLM (via Ollama) to output layer-wise priors—Xavier-style Normal distributions for convolutional weights and delta priors for biases—in JSON form. These distributions are parsed and applied to each Conv2d and Linear layer, effectively treating the LLM’s responses as a “knowledge likelihood” that narrows the network’s search space. Training then proceeds on CIFAR-10 with strong augmentation (random crops, flips, rotations, color jitter, random erasing), using AdamW with cosine-annealed learning rates and weight decay. By starting closer to a sensible solution, the LLM-informed initialization speeds up convergence and boosts test accuracy compared to a purely random start.

5.2.3 Experiment 3: IMDB Sentiment with and without LLM-Elicited Priors

🔗 We then tried using LLM-elicited priors in the same LSTM by prompting a language model for sentiment scores and using them to bias embedding initialization and forget-gate settings. Surprisingly, this informed setup only reached around 83.08% accuracy—lower than the 87% baseline. This drop suggests that directly injecting LLM priors might misalign with the LSTM’s internal learning, especially if the priors aren’t context-aware or are too simplistic. To see real gains, we may need better-designed prompts or smarter ways to translate LLM knowledge into model parameters.

5.3 Learnings

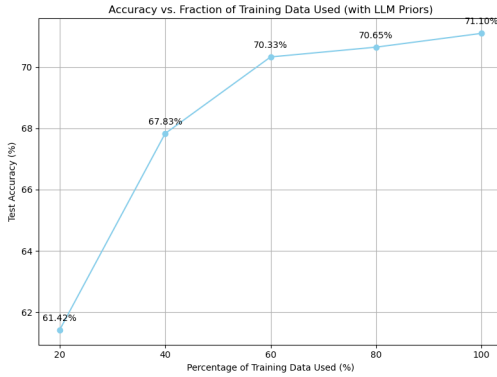
Based on the outcomes of our three studies—(1) training a vanilla CNN from scratch on CIFAR-10, (2) enhancing that CNN with AutoElicit’s LLM-derived priors, and (3) evaluating an LSTM sentiment classifier on IMDB both with and without AutoElicit priors—we can draw insights into how structured, model-agnostic priors and synthetic knowledge affect deep learning performance. The following key findings are supported by our quantitative results:

a) **Baseline CNN Performance (No Priors):** Training our SimpleCNN on just 20% of CIFAR-10 yielded a test accuracy of **61.42%**, which rose to **71.10%** when using the full 100% training set.

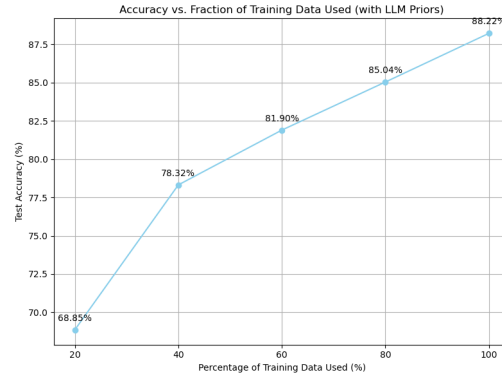
Notably, gains tapered off beyond 60% of the data—improving only from 70.33% at 60% to 70.65% at 80%, and finally to 71.10% at 100%. This diminishing return suggests that merely increasing dataset size, without enhancing model capacity, regularization, or augmentation, yields limited benefits.

b) LLM-Elicited Priors on CIFAR-10: When we initialized our CNN with AutoElicit’s LLM-derived Xavier priors, test accuracy jumped from **68.85%** using only 20% of the data to **88.22%** when trained on the full 100%. Compared to the vanilla baseline, this represents a ~ 7 pp gain in the low-data regime (20%) and an ~ 18 pp boost at 100%. Moreover, accuracy improvements were more consistent as we increased data—rising roughly 10 pp from 20% to 40%, then tapering more gently—highlighting that LLM priors both accelerate early learning and elevate final performance by steering the network toward meaningful feature detectors from the outset.

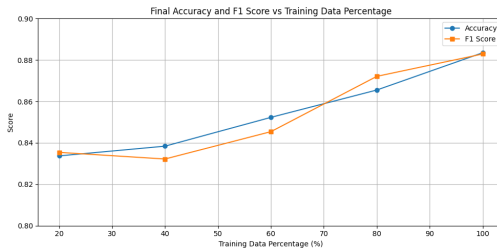
c) IMDB Sentiment with and without LLM-Elicited Priors: On the IMDB review task, our baseline LSTM reached about 87% accuracy after tuning. But when we added LLM-elicited priors—like using average polarity scores for embeddings and setting forget-gate biases—the accuracy unexpectedly dropped to around 83.08%. This suggests that simply plugging in LLM-derived priors might disrupt the LSTM’s learning, especially if the priors aren’t well-aligned or context-aware. It highlights the need for more thoughtful integration of LLM knowledge, possibly through better prompts or smarter mapping strategies.



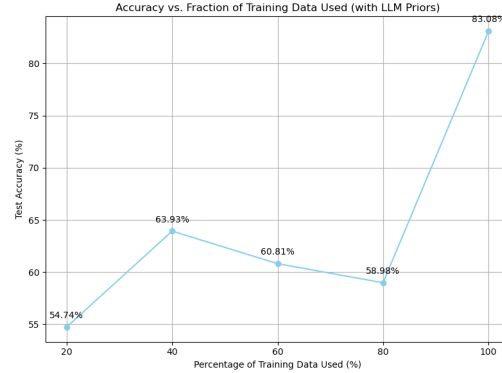
(a) Experiment 1: Without LLM elicited Priors



(b) Experiment 2: With LLM elicited Priors



(c) Experiment 3: Without LLM elicited Priors



(d) Experiment 3: With LLM elicited Priors

Figure 3: Accuracy vs fraction of training data used with and without LLM elicited priors.

5.4 Limitations and future research

While using LLM-generated priors (via AutoElicit) showed clear benefits in vision models like CNNs on CIFAR-10, their impact was limited in the IMDB sentiment classification task with LSTMs. Specifically, attempts to initialize word embeddings or gate biases using sentiment-informed priors did not lead to performance improvements over random initialization. This highlights a key limitation: direct prior injection from LLMs may work well in structured, low-level feature spaces (e.g., filters in CNNs) but is less effective in complex, high-dimensional linguistic spaces without more tailored integration strategies.

Future work could explore more effective ways to align LLM-generated priors with neural network architectures, especially in NLP. This might involve using task-specific prompt engineering, refining priors during training (e.g., via gradient feedback), or combining LLM insights with uncertainty-based sampling in active learning. Additionally, extending AutoElicit to larger-scale models, more diverse tasks, and real-world data scenarios could help validate its generalizability and inform new methods for prior elicitation grounded in model behavior.

References

- [1] Capstick, A., Krishnan, R.G., & Barnaghi, P. (2023). *AutoElicit: Using Large Language Models for Expert Prior Elicitation in Predictive Modelling*. arXiv preprint arXiv:2307.04609.
- [2] Seung, H.S., Oppor, M., & Sompolinsky, H. (1992). *Query by Committee*. In Proceedings of the Fifth Annual Workshop on Computational Learning Theory, pp. 287–294. ACM.
- [3] Liu, S. & Li, X. (2022). *Understanding Uncertainty Sampling in Active Learning*. In Proceedings of the 39th International Conference on Machine Learning (ICML), pp. 13487–13497.

Appendix

The repository of codes used in this paper can be found here : <https://github.com/chaitanyavb-502/CS-772-project/blob/main/README.md>